

Programmation procédurale : fonctions et tableaux



On parle de quoi ?

1. [Passage par copie \(rappel\)](#).
2. [Passage par référence](#)
3. [Tableau en entrée](#)
4. [Tableau en sortie](#)
5. [En résumé...](#)

Passage par copie (rappel)

En C, le passage d'une variable contenant un type primitif en argument d'une fonction se fait **toujours par copie**.

→ lors de l'appel de la fonction, une copie des arguments est passé à la fonction

Passage par copie (rappel)

```
void ajouteDix(int n);

void main (void){
    int monEntier = 10;
    ajouteDix(monEntier);
    printf("%d", monEntier); // monEntier = 10
}

void ajouteDix(int n){
    n = n + 10;
}
```

monEntier

type : int

10

0x7fff0a00ecc0

n

type : int

20

0x7ffe7dcce490

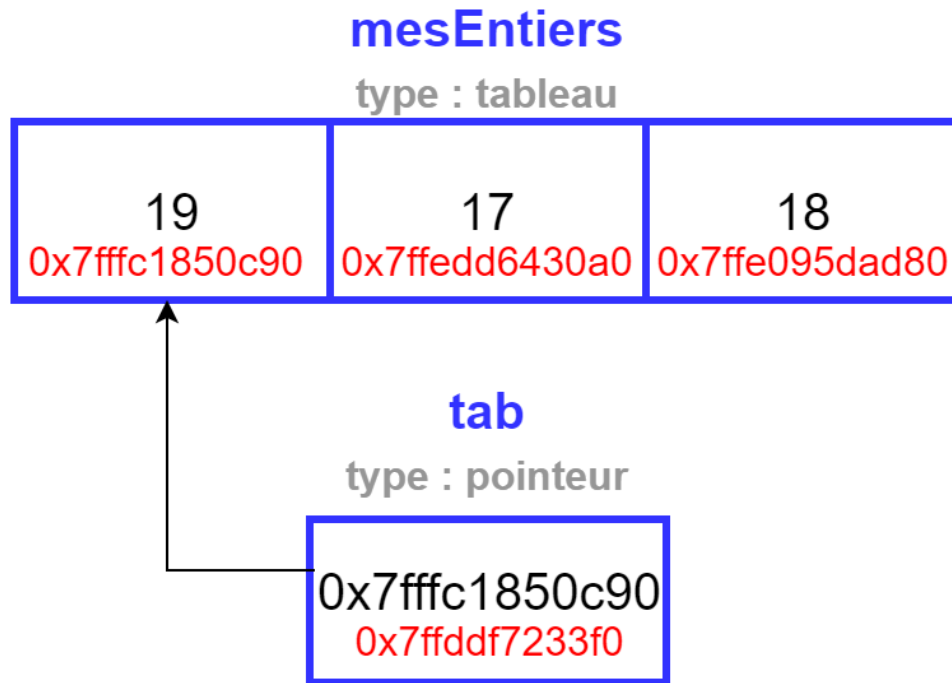
Passage par référence

Un tableau passé en argument d'une fonction est toujours passé par **référence**.

La référence (l'adresse) d'un tableau correspond toujours à l'adresse du premier élément du tableau.

Passage par référence

```
void main(void){  
    int mesEntiers[] = {19,17,18};  
    imprimePremier(mesEntiers);  
}  
  
void imprimePremier(int tab[]){  
    printf("%d\n", tab[0]);  
}
```



Passage par référence

```
void testRef(int tab[]);
```

```
void main(void){
```

```
    int tab[] = {19,17,18};
```

```
    printf("La première case se trouve à l'adresse %p\n", &tab[0]);
```

```
    testRef(tab);
```

```
}
```

```
void testRef(int tab[]){
```

```
    printf("Le pointeur est stocké à l'adresse %p\n", &tab);
```

```
    printf("Le pointeur pointe vers l'adresse %p\n", tab);
```

```
}
```

Passage par copie vs. référence

pass by reference



fillCup()

pass by value



fillCup()

www.penjee.com

Tableau en entrée : un cas concret

On vous demande de créer une fonction qui calcule l'age moyen d'une classe de 10 élèves.

Cette fonction recevra en entrée un tableau contenant 10 entiers représentant chacun l'age d'un élève.



```
float ageMoyen (int ages[])
```

Tableau en entrée : un cas concret

Solution

Cette solution ne fonctionne pas. Pourquoi ?

```
float ageMoyen(int ages[]){  
    int taille = sizeof(ages);  
    int somme = 0;  
    for (int i = 0; i < taille; i++){  
        somme += ages[i];  
    }  
    return (float) somme / taille;  
}
```

Tableau en entrée : un cas concret

Solution

Étant donné que les tableaux sont passés par référence, ce que la fonction `ageMoyen` reçoit, ça n'est pas le tableau mais sa **référence** (son pointeur).

→ Cette référence est en réalité l'adresse de la première case du tableau `ages`.

Pour résoudre le problème, on passe la taille du tableau comme argument supplémentaire.



```
float ageMoyen (int ages[], int taille)
```

Tableau en entrée : un cas concret

Solution

```
void main(void){  
    int ages[] = {18,17,19,18,17,16,16,17,18,17};  
    printf("Moyenne : %.2f", ageMoyen(ages, sizeof(ages)/sizeof(int)));  
}  
  
float ageMoyen(int ages[], int taille){  
    int somme = 0;  
    for (int i = 0; i < taille; i++){  
        somme += ages[i];  
    }  
    return (float) somme / taille;  
}
```

Tableau en entrée : un cas concret

Solution

On peut également utiliser les constante symbolique :

```
#define NB_ELEVES 10

float ageMoyen(int ages[]);

void main(void){
    int ages[NB_ELEVES] = {18,17,19,18,17,16,16,17,18,17};
    printf("Moyenne : %.2f", ageMoyen(ages));
}

float ageMoyen(int ages[]){
    int somme = 0;
    for (int i = 0; i < NB_ELEVES; i++){
        somme += ages[i];
    }
    return (float) somme / NB_ELEVES;
}
```

Tableau en sortie : un cas concret

On vous demande de créer une fonction qui initialise à 0 un tableau de 10 entiers.



? `initialisation (int ages[], int taille)`

Tableau en sortie : un cas concret

Un tableau en sortie se fait toujours **via les arguments de la fonction**.

En effet, puisqu'il est passé par référence (et pas par copie), on peut directement le modifier dans la fonction.



```
void initialisation (int ages[], int taille)
```

Tableau en sortie : un cas concret

Solution

```
#define TAILLE_TAB 10
void initialisation(int tab[]);

void main (void){
    int tab[TAILLE_TAB];
    initialisation(tab, TAILLE_TAB);
}

void initialisation(int tab[], int taille){
    for(int i = 0; i < taille; i++){
        tab[i] = 0;
    }
}
```


En résumé...

- **Un type primitif** est toujours **passé par copie** à une fonction
- **Un tableau** est toujours **passé par référence** à une fonction
- Lorsqu'un tableau est passé en argument à une fonction, il faut obligatoirement ajouter **un argument supplémentaire** représentant la taille du tableau